

Apple Detection and Counting Using Neural Networks

Roxana Garderes

Facultad de Ingeniería
Universidad de la República
Montevideo, Uruguay
roxana.garderes@fing.edu.uy

Facundo Gutiérrez

Facultad de Ingeniería
Universidad de la República
Montevideo, Uruguay
facundo.gutierrez@fing.edu.uy

Mercedes Marzoa Tanco

Facultad de Ingeniería
Universidad de la República
Montevideo, Uruguay
mmarzoa@fing.edu.uy

Gonzalo Tejera

Facultad de Ingeniería
Universidad de la República
Montevideo, Uruguay
gtejera@fing.edu.uy

Abstract—It is important to estimate the number of fruits which are going to be harvested, since that information allows farmers to make key decisions regarding the production, distribution and commercialization of crops. In this project, solutions for detecting and counting apples are explored, by using detection and tracking algorithms based on neural networks. Four neural network object detection models were trained: YOLOv5, YOLOv8 and two variants of Faster R-CNN, using public as well as self-made datasets. Three tracking algorithms (StrongSort, ByteTrack and OCSort) were tested using the trained models and labeled video datasets. These datasets were developed with recordings of a real-world orchard and recordings generated with a ROS-based apple field simulator, developed as part of this project. Finally, two fitting models based on linear regression were evaluated in order to minimize prediction errors. As a conclusion of this research, it was found that the YOLOv5 and YOLOv8 performances were better than the other two, achieving a mAP50 of 0.87 and 0.85. In terms of apple tracking, the performance of OCSort stands out, reaching a HOTA metric of 60.24% and running times in the order of 30 ms.

Index Terms—Apples, Neural Networks, Counting, Detection, Tracking, ROS (Robot Operating System), YOLOv5, YOLOv8, Faster R-CNN, SORT, OCSort, ByteTrack, StrongSort, Linear Regression

I. INTRODUCCIÓN

El concepto de agricultura inteligente ha tomado relevancia en los últimos años. Las redes neuronales, en particular las redes neuronales convolucionales (CNN), han mostrado un gran potencial en la resolución de problemas complejos, incluso en problemas vinculados a la agricultura [1]. Además, las CNN han evolucionado notablemente, convirtiéndose en sinónimo de excelencia en términos de resultados, realizando tareas de clasificación y predicción, y superando con creces a técnicas clásicas de procesamiento de imagen.

Los avances en esta área podrían tener importantes aplicaciones en la optimización de la producción agrícola, ya que un conocimiento anticipado de la cantidad de manzanas a cosechar permite una mejor planificación de ventas, así como de la mano de obra y recursos para la cosecha, traslado y distribución de las manzanas, llevando a una reducción de los costos, mayor exactitud en la planificación y fomentando una producción sustentable [2], [3].

La identificación y conteo de frutas en plantaciones representa un gran desafío debido a la variedad de tamaños, colores y formas que pueden presentar las frutas, y las oclusiones por otras manzanas, ramas y hojas. Trabajar en ambientes no controlados, como lo es una plantación de manzanos, conlleva variables que pueden afectar notoriamente el resultado. El clima, la hora del día y las oclusiones de los frutos, condicionan el desempeño de los métodos elegidos. Factores como el recorrido elegido para grabar los datos impactan en la visibilidad de filas de árboles contiguas, lo que puede alterar considerablemente los resultados de conteo.

Otro problema frecuente a la hora de utilizar redes neuronales para estimar cantidades de manzanas en árboles, es el contar con los datos suficientes para entrenarlas. Si bien existen conjuntos de datos públicos de imágenes de manzanas, que son muy útiles para abordar el problema de detección, no fue posible encontrar grabaciones en video que contengan filas completas de manzanas, las cuales son necesarias para evaluar soluciones de conteo.

Los simuladores son relevantes en inteligencia artificial, y más aún cuando se utilizan robots, pues proporcionan un entorno controlado y escalable para entrenar modelos, explorar una variedad de escenarios y reducir costos y riesgos asociados con el desarrollo y despliegue en el mundo real. En virtud de estas consideraciones, se relevaron características de los manzanos y sus montes, y se desarrolló un simulador parametrizable que permite exportar videos del recorrido de un robot terrestre.

A lo largo del trabajo se entrenaron y compararon cuatro modelos de redes neuronales, presentando mejores resultados YOLOv5 [4] y YOLOv8 [5]. Estas últimas se utilizaron junto con algunos algoritmos de seguimiento del estado del arte (OCSort [6], ByteTrack [7] y StrongSort [8]) para detectar y seguir las frutas en una única fila de manzanos. Idealmente, la cantidad de identificadores asignados corresponde a la cantidad de manzanas en la fila.

Sin embargo, por problemas como la visibilidad variable de las manzanas, la cantidad requiere ser ajustada para acercarse más a la cantidad esperada de manzanas. Para esto se plantea un ajuste basado en regresión lineal.

Este informe inicia con una revisión de la literatura que abarca estudios previos pertinentes al tema en cuestión, luego

se detallan las redes neuronales y algoritmos de seguimiento seleccionados. A continuación se exponen los *datasets* a utilizar, incluyendo la creación de un simulador de campos de manzanas. Finalmente, se describen las pruebas ejecutadas en la experimentación y se analizan los resultados obtenidos.

II. TRABAJOS RELACIONADOS

En esta sección se incluyen algunos proyectos que presentan similitudes con el que se describe en esta publicación. Por la diferencia de tecnologías y métricas utilizadas, se dividen en dos apartados, diferenciando investigaciones de detección con las que incluyen conteo.

II-A. Detección

En lo que respecta a la aplicación de redes neuronales para resolver el problema de detección de manzanas, se destacan los resultados de dos modelos, Faster R-CNN y YOLO, en sus diferentes versiones.

En [9] se propone el uso del modelo Faster R-CNN como sistema de detección de frutas, concretamente almendras, mangos y manzanas. Estas tres frutas exhiben notables diferencias morfológicas, tales como la cantidad de frutos por árbol, el color y la textura, lo cual plantea un desafío significativo.

El entrenamiento de la RN se realiza con imágenes tomadas durante el día en distintas plantaciones, utilizando un robot terrestre y técnicas de *data augmentation*. Con esto evalúan dos modelos de Faster R-CNN, logrando obtener $F1$ de 0.876 y 0.908.

Otro ejemplo relevante es la utilización de YOLOv5 para reconocer las manzanas que pueden ser recolectadas por un robot [10]. Para esto se deben detectar las manzanas y clasificarlas entre las que puedan ser recogidas directamente por el robot y las que presentan algún obstáculo que lo impida (como una rama u otra manzana). En este caso, la elección del algoritmo YOLOv5 se debe a que puede ser utilizado en tiempo real, es rápido de entrenar y ligero, por lo que es razonable de implementar en dispositivos embebidos.

Se tomaron 1214 imágenes de manzanas para el entrenamiento. Estas fueron obtenidas de una plantación, en diferentes horarios y condiciones lumínicas, a distancias de entre 0.5 y 1.5 metros. El modelo entrenado alcanza valores de mAP de 86.75 %, P de 91.48 % y $F1$ de 87.49 %, con un tiempo de detección promedio de 0.013 segundos por imagen.

II-B. Conteo

A la hora de contar objetos en video, el conteo de duplicados es uno de los problemas más frecuentes [11]. Una manzana puede ser cubierta con otro objeto por algunos segundos y luego volver a estar visible, o ser visible desde ambos lados de la fila de árboles, llevando a ser contada más de una vez.

Para este desafío se encuentran varias propuestas de soluciones, que combinan diferentes tecnologías. Un ejemplo de esto es la construcción de modelos 3D de campos de manzanas a partir de los datos obtenidos de ambos lados de una hilera de manzanos, unificando los modelos construidos con los datos de cada lado [12].

Otro ejemplo es una aplicación de Faster R-CNN [13], en la cual se combina la detección con el seguimiento de las frutas a lo largo de los cuadros para lograr la identificación individual y, por consiguiente, el conteo preciso. Con este fin, se utilizan puntos proporcionados por el LiDAR para la ubicación GPS de cada árbol, y las frutas se asocian con su respectivo árbol.

También se encuentran ejemplos en la bibliografía de combinaciones de algoritmos de detección con seguimiento, que utilizan video como única entrada de datos. En [11] se combinan y personalizan YOLOv3 y SORT para detectar y contar cítricos, logrando un AP de 0.938 en detección y un error de entre 6 % y 40 % en conteo.

III. METODOLOGÍA

Este estudio utiliza tanto conjuntos de datos públicos como de elaboración propia. Con estos datos, se entrenaron y probaron cuatro modelos de redes neuronales (RN) para detección de manzanas en árboles.

Primero se seleccionaron los mejores modelos de cada RN de acuerdo a los resultados obtenidos sobre el conjunto de *test* de cada *dataset*. Luego, estos modelos entrenados se ejecutaron sobre un *dataset* diferente, diseñado específicamente para evaluarlos.

La siguiente etapa del proyecto consistió en valorar los resultados de los algoritmos de seguimiento seleccionados, utilizando los dos mejores modelos de detección entrenados de la etapa anterior. Dicha evaluación se realizó con un conjunto de datos consistente en cuadros de video etiquetados y utilizando métricas del Mot Challenge [14].

Por otro lado, se realizaron pruebas de conteo utilizando las diferentes combinaciones de RN de detección y algoritmos de seguimiento, utilizando videos de recorridos completos de una fila de manzanos, de la cual se conocía la cantidad total de manzanas recolectadas momentos después de filmado el video.

Finalmente, se analizaron dos modelos de ajuste de los resultados de conteo obtenidos. Se generaron dos métodos, uno aplicando regresión lineal y otro mediante la generación de un coeficiente. Ambos modelos se construyeron utilizando datos de cantidad real de manzanas y resultados obtenidos sobre videos de instancias simuladas.

III-A. Selección de modelos

A nivel de RN para detección, se seleccionaron modelos de YOLO y de Faster-RCNN, con el fin de comparar arquitecturas diferentes del estado del arte.

Las implementaciones de YOLO seleccionadas para la parte experimental fueron YOLOv5 y YOLOv8. La elección de YOLOv5 se dio por ser el estado del arte con aplicaciones en detección de manzanas, con respaldo de la comunidad y múltiples aplicaciones documentadas. En particular, se escogió el modelo *yolov5s.yaml* (*small*), por ser el más liviano de los disponibles en el 2022.

La selección de YOLOv8 se debió al buen desempeño de este algoritmo y porque incluía la posibilidad de ejecutar seguimiento nativo. Para esta implementación se seleccionó

el modelo *YOLOv8n (nano)*, por ser el más liviano y rápido de entrenar y ejecutar.

Por otro lado, se observó que la librería Detectron2 [15], desarrollada por Facebook, ofrece una forma más simple e intuitiva de utilizar Faster R-CNN que en las implementaciones oficiales. Detectron2, implementado en Python y utilizando la librería Pytorch, proporciona una interfaz sencilla que facilita la implementación, entrenamiento y evaluación de algoritmos de detección y segmentación, entre los que destaca Faster R-CNN.

En lo que respecta a Faster R-CNN, Detectron2 provee una serie de modelos preentrenados con COCO [16]. A partir de los resultados disponibles, se seleccionaron los modelos con mejor desempeño en cuanto a tiempo de inferencia y medida *mAP* sobre COCO:

1. *faster_rcnn_R_50_FPN_3x*: con *mAP* de 40.2 % y un tiempo de inferencia de 0.038 segundos.
2. *faster_rcnn_X_101_32x8d_FPN_3x*: con *mAP* de 43.0 % y un tiempo de inferencia de 0.098 segundos.

Donde *R_50_FPN* y *X_101_32x8d_FPN* son las redes *backbone* y *3x* es la cantidad de ciclos de entrenamiento sobre el conjunto COCO. En adelante, a estos modelos se les nombrará como *Faster-50* y *Faster-101* respectivamente.

III-B. Algoritmos de seguimiento

Para ejecutar los algoritmos de seguimiento, se seleccionó [17] por permitir la ejecución con varias redes neuronales de detección y algoritmos de seguimiento, así como la posibilidad de exportar métricas con el formato de MOT Challenge [18]. Dicho proyecto incluye como algoritmos de seguimiento OC-Sort, ByteTrack y StrongSort, considerados suficientes para la etapa de experimentación debido a sus numerosas diferencias.

III-C. Conjuntos de datos

III-C1. Simulador de campos de manzanas: Contar con un simulador que genere datos para entrenar modelos de detección y conteo es de gran utilidad, ya que posibilita simular distintos ambientes conociendo exactamente datos como la cantidad total de las manzanas y sus coordenadas. A su vez, permite tener control sobre distintos factores externos, como la iluminación o el clima, y generar datos sin importar la etapa de maduración de la fruta a estudiar.

Esto motivó el desarrollo de un simulador basado en ROS [19], que permite generar campos de manzanas aleatorios a partir de una serie de parámetros configurables. Para esto fue necesario conocer las características morfológicas y de desarrollo de los manzanos. A partir de una serie de reuniones con expertos en el área y bibliografía recomendada [20], se lograron relevar los requisitos necesarios para la implementación del simulador.

Dichos requisitos definen ciertas condiciones esperadas para los árboles de manzanas, como su altura (2.3m a 3.5m), el grosor de las ramas principales y secundarias (un máximo de 14cm y 2cm respectivamente), el tamaño de las frutas (entre 6.5 y 7.0cm en promedio) y de las hojas (un máximo de 14cm).

A nivel de distribución, los árboles se ubican en filas cada 1m y las filas separadas por una distancia de entre 2.5 y 4m.

Esta implementación parte del proyecto Fields Ignition [21], desarrollado con el objetivo de generar ambientes aleatorios de pruebas para robots agrícolas. Utilizando Ignition Gazebo [22] y la API de Python provista por Blender [23], el simulador permite generar campos aleatorios de tomates y realizar recorridos en un entorno simulado.

Si bien los resultados provistos por el simulador son satisfactorios y de gran utilidad a la hora de realizar un trabajo de investigación de esta índole, vale la pena aclarar que se trata de una herramienta complementaria que ignora una gran cantidad de aspectos del mundo real (iluminación, texturas, irregularidades del camino y clima, entre otras). Por lo tanto, no sustituye la validación en un entorno real.

III-C1a. Generación de los árboles: Los árboles de manzanas pueden tener distintas morfologías según el tipo de plantación [20]. Un árbol pueden tener una o más ramas principales, donde nacen ramas secundarias. En las ramas secundarias crecen brindillas, en las que brotan las hojas y las manzanas.

La generación del árbol en el simulador consiste en la ejecución de un *script* en python que utiliza la API de Blender, y se puede resumir en dos instancias. En la primera se genera el esqueleto del árbol, es decir, la estructura de ramas principales y ramas secundarias, que da como resultado un modelo similar al de la figura 1a. Posteriormente, se agregan de forma aleatoria las hojas y frutas en las ramas secundarias, lo que da como resultado un árbol completo como el que se muestra en la figura 1b.

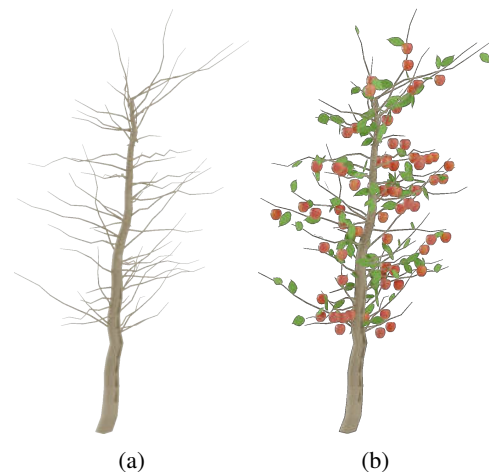


Figura 1: Ejemplo de un árbol (1b) y esqueleto del árbol (1a) generado por el simulador

La generación del árbol es parametrizable, pudiendo definir el tamaño de las frutas, la longitud, la cantidad de ramas principales y las distintas probabilidades para agregar frutas y hojas en las ramas secundarias.

III-C1b. Generación del mundo simulado y ejecución: Para generar cada instancia se necesitan cuatro parámetros

clave: la cantidad de filas (n), la cantidad de árboles por fila (m), la distancia entre árboles y la distancia entre filas. A partir de estos parámetros, se generan los $n \times m$ árboles siguiendo el proceso detallado en la sección anterior.

El modelo de robot elegido es el robot Husky [24], que incluye sensores de navegación, LiDAR y cámara RGB. El modelo original se modifica de forma que la posición de la cámara sea tomada como parámetro de la generación de la instancia, y se elimina el modelo 3D que representa el soporte de los sensores, a modo de evitar posibles interferencias con la cámara. Además, los parámetros de configuración de la cámara se modifican para simular uno de los lentes de las cámaras ZED 2 [25], con el propósito de replicar con mayor precisión el ambiente real.

El modelo de suelo consiste en una superficie cuadrada modelada en un objeto DAE, al cual se le asignan físicas de colisión para evitar que los objetos pasen a través de él. Respecto al modelo de suelo original, cambia únicamente la escala y la textura, pasando de simular un suelo de tierra a pasto.

La ejecución de cada instancia simulada se puede traducir a la arquitectura definida en la figura 2. Ignition interpreta el esquema definido en el archivo SDF que define la instancia y publica los tópicos de Ignition correspondientes, los cuales son traducidos a mensajes de ROS, que se publican en tópicos de ROS. Entre los mensajes publicados se encuentran mensajes de navegación, de la cámara y de otros sensores como el LiDAR.

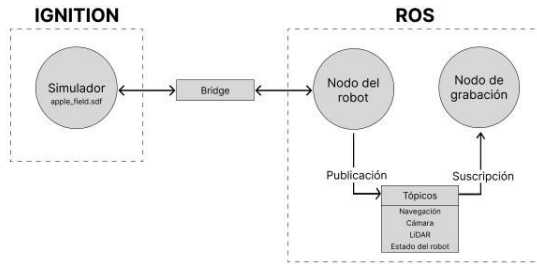


Figura 2: Arquitectura del simulador.

Un ejemplo de la ventana de Ignition ejecutando una instancia del simulador se puede observar en la figura 3.

III-D. Conjuntos de datos utilizados

III-D0a. Datos para detección: Para entrenar y evaluar los algoritmos de detección se utilizaron 5 conjuntos de datos, seleccionados por diferentes motivos. Algunos de estos conjuntos de datos son públicos, extraídos de distintas fuentes, mientras que otros fueron elaborados como parte de este proyecto. Parte de estos últimos fueron tomados en un campo de experimentación agrícola del Instituto Nacional de Investigación Agropecuaria (INIA), en Uruguay.

- **Dataset 1 (DS1):** obtenido de la plataforma Universe de Roboflow [26]. Consiste en 232 imágenes etiquetadas de árboles de manzanas, donde las manzanas parcialmente ocluidas no son etiquetadas y el área etiquetada es mayor al área de la manzana.

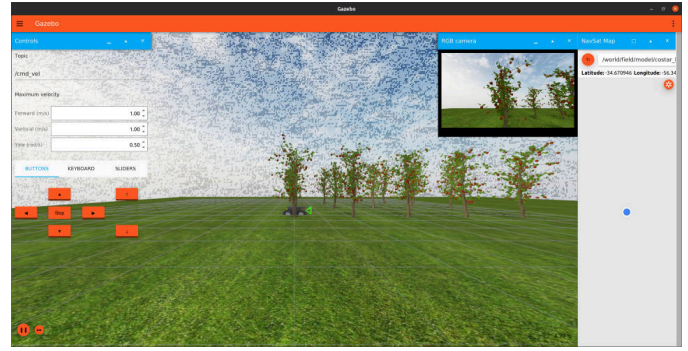


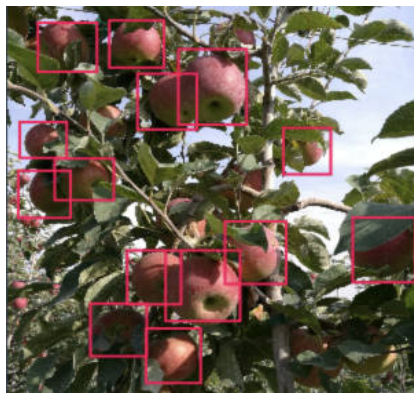
Figura 3: Ventana de Ignition con una instancia de simulador ejecutando.

- **Dataset 2 (DS2):** obtenido también de la plataforma Roboflow [27], consiste en 1789 imágenes de árboles de manzanas. Incluye las imágenes del conjunto de datos Minneapple [28], así como imágenes desde una vista aérea por encima de los árboles, y primeros planos de manzanas.
- **Dataset 3 (DS3):** también obtenido de Roboflow [29], consiste en 331 imágenes etiquetadas de árboles de manzana.
- **Dataset 4 (DS4):** de elaboración propia, consiste en 80 imágenes obtenidas en una salida de campo a INIA Las Brujas. Las imágenes fueron capturas con un dispositivo Xiaomi Redmi Note 11 Pro 5G y los árboles capturados están parcialmente cubiertos por una red para aves, lo que dificulta el proceso de etiquetado y detección.
- **Dataset 5 (DS5):** también de elaboración propia, contiene 48 imágenes obtenidas en una salida de campo. A diferencia del conjunto de datos DS4, los árboles de la fila no tienen la red para aves.
- **Dataset 7 (DS7):** corresponde al conjunto de datos MinneApple [28]. Consiste en 41000 manzanas etiquetadas en un total 1000 imágenes tomadas a lo largo de dos años.

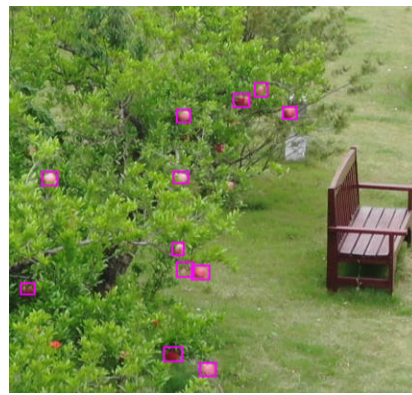
En la figura 4 se pueden observar imágenes representativas de cada conjunto de datos.

Los conjuntos de datos para entrenar los modelos se complementaron con técnicas de *Data Augmentation* que se detallan a continuación, con sus nombres correspondientes en la plataforma [30].

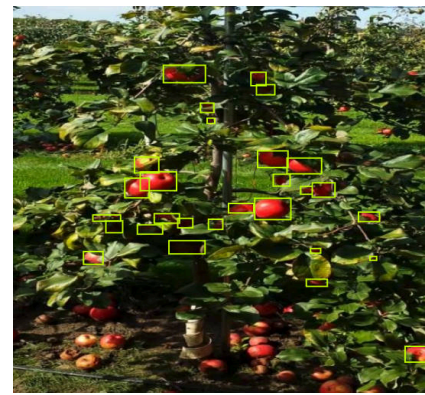
- **Corte (crop):** se selecciona una parte de la imagen aleatoriamente, resultando un equivalente a entre 0 % y 20 % de zoom.
- **Rotación (rotation):** rotación aleatoria de las imágenes entre -15° y +15°.
- **Saturación (saturation):** ajuste aleatorio de la saturación de una imagen entre -40 % y +40 %, para hacerla más brillante o más oscura.
- **Brillo (brightness):** ajuste del brillo de una imagen de entre -50 % y +50 %.
- **Exposición (exposure):** ajuste aleatorio de la exposición



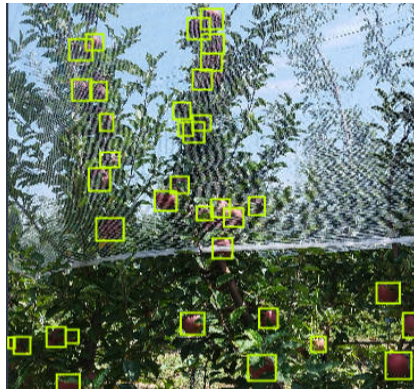
(a) Dataset 1



(b) Dataset 2



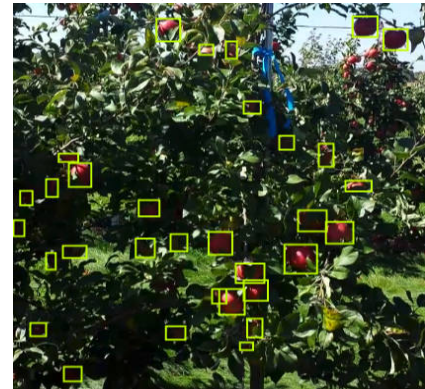
(c) Dataset 3



(d) Dataset 4



(e) Dataset 5



(f) Dataset 7

Figura 4: Imágenes representativas de cada *dataset*.

gama de una imagen entre -15 % y +15 %, para hacerla más brillante o más oscura.

- Desenfoque (*blur*): agregado de *gaussian blur* de hasta 1px.

III-D1. Datos para seguimiento: Con el fin de evaluar los modelos de detección se contaba con dos videos, uno generado con el simulador y otro filmado con un celular *Samsung Galaxy S21 FE* sujeto a un robot Jackal [31], de una fila de manzanos en INIA Las Brujas, recorriendo la fila en un sentido de un lado de la fila y en el sentido opuesto del otro lado.

El video filmado en INIA Las Brujas se dividió en dos partes, según el lado de la fila (ida y vuelta). Se etiquetaron alrededor de 200 cuadros en cada parte (procurando que fueran el mismo fragmento de la fila) utilizando la plataforma *Supervisely* [32]. Además, se sabe que en la fila se recolectaron 2133 manzanas al final del día, aunque no se sabe exactamente cuántas aparecen en el video.

En resumen, los conjuntos de datos para seguimiento son:

- DS-SIM: generado con el simulador, etiquetado, tiene una duración de 6 segundos y está compuesto por una única fila de 4 de árboles, donde aparecen 61 manzanas.
- DS-INIA-1: fragmento del video de ida de la filmación de la fila en campo, etiquetado, con una duración de 20 segundos y 139 manzanas.

- DS-INIA-2: fragmento del video de vuelta de la filmación de la fila en campo, etiquetado, con una duración de 25 segundos y 200 manzanas.
- DS-INIA-IDA: primera mitad del recorrido de la fila en campo (de ida), sin etiquetar, de 2 minutos 38 segundos.
- DS-INIA-VUELTA: segunda mitad del recorrido de la fila en campo (de vuelta), sin etiquetar, de 2 minutos 39 segundos.
- DS-INIA-COMPLETO: recorrido completo de la fila (ida y vuelta, con el giro intermedio), sin etiquetar, de 5 minutos 45 segundos.

IV. EXPERIMENTACIÓN

La etapa de experimentación se dividió fundamentalmente en dos: el entrenamiento y selección de redes neuronales para detección de manzanas, y el uso de estas redes entrenadas en conjunto con algoritmos de seguimiento para el rastreo y conteo de manzanas en video. Se documentan por separado por utilizar conjuntos de datos, métricas y algoritmos diferentes.

Para garantizar consistencia y disponibilidad de recursos, el entrenamiento y evaluación de los modelos se realizó en la plataforma *Google Colab* [33]. En el caso particular de los entrenamientos, considerando los altos tiempos de ejecución, se utilizó el plan *Pro* de dicha plataforma, mientras que para la evaluación de los algoritmos de detección y seguimiento, el plan gratuito fue suficiente.



(a) Video simulado



(b) Video filmado

Figura 5: Ejemplos de cuadros etiquetados en Supervisely

IV-A. Detección

Los cuatro modelos de detección seleccionados (YOLOv5, YOLOv8, Faster-101 y Faster-50) fueron entrenados con los conjuntos de datos descritos en la sección III-C con una distribución de 70 % para entrenamiento, 20 % para validación y 10 % para pruebas.

La métrica comúnmente utilizada para evaluar modelos de detección de objetos es *Mean Average Precision (mAP)* [34], que se calcula como la media de la métrica *Average Precision (AP)* por cada clase que se busca detectar (en este caso es una única clase, manzana). La métrica *AP* se calcula como el área bajo la curva *Precision – Recall*, por lo que el umbral de *IoU* elegido impacta directamente en el valor de esta métrica. El umbral más popular en la literatura es 50 %, lo que indica que al menos un 50 % del objeto detectado coincide con el objeto real.

Las épocas o *epochs* son los ciclos de entrenamiento de una red neuronal, y este fue el parámetro seleccionado para variar en esta instancia de experimentación por considerarse clave y altamente influyente. En general, a mayor cantidad de épocas, la red obtiene mejores resultados, pero también tarda más el entrenamiento y puede llevar a sobreajustes [35].

Para determinar cuál era la cantidad óptima de épocas para entrenar las RN en este caso, se realizaron entrenamientos de los mismos modelos con conjuntos de datos variando la cantidad de épocas. Inicialmente, se tomaron 50 y 100 épocas [36], pero luego se agregaron 20 épocas para las RN más pesadas, así como 200 y 500 para las más livianas.

No se realizaron pruebas con mayor cantidad de épocas porque los resultados indicaban que no existían mejoras en los modelos entrenados (en el mejor caso, a partir del ciclo 151, pero en general mucho antes), lo cual se había observado previamente en la bibliografía [10]. A modo de referencia, se incluyen las gráficas resultantes del entrenamiento de YOLOv8 con el DS2 para 200 épocas en la figura 6.

Para todas las redes entrenadas, los mejores resultados se dan con el DS2. Esto era esperable, ya que es el *dataset* de mayor cantidad y variedad de imágenes. En el cuadro I se

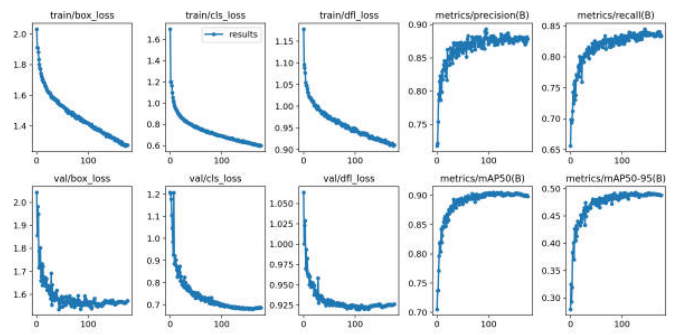


Figura 6: Resultados del entrenamiento de YOLOv8 con DS2 para 200 épocas.

Modelo	Épocas	AP_{50}
YOLOv5	50	0.906
YOLOv8	200	0.904
Faster-101	20	0.869
Faster-50	50	0.878

Cuadro I: Resultados de AP_{50} por modelo de detección y épocas para el DS2.

incluyen los mejores resultados de cada red neuronal para dicho *dataset*, detallando las épocas y el AP_{50} obtenido.

Por otro lado, analizando los resultados respecto a las épocas y la cantidad de imágenes, se observa que para los conjuntos de datos grandes (a partir de 1000 imágenes), los entrenamientos con 50 épocas dan resultados ligeramente mejores que en 100 o 200, indicando que no existen mejoras a partir de las 50 épocas. En cambio, en los *datasets* más pequeños se obtienen mejores resultados con 100 y 200 épocas.

A su vez, se observa que Faster-101 logra resultados similares a los de YOLOv5 y YOLOv8 en 20 épocas, pero el tiempo requerido para entrenar el modelo es considerablemente mayor, y no resultaba viable entrenarlo con más de 20. Lo mismo ocurrió con Faster-50, donde se pudo entrenar solo con 20 y 50 épocas.

IV-A1. Evaluación sobre DS5: Como se mencionó anteriormente, DS5 es un conjunto de datos etiquetados a partir de imágenes capturadas en la plantación objetivo de este proyecto, e incluso muchas de estas imágenes son cuadros de los videos utilizados para probar en la sección de seguimiento y conteo. Por este motivo, la validación más relevante de los modelos entrenados es sobre este conjunto de datos, ya que cuenta con características ambientales y espaciales más similares a las que van a ser utilizadas en las pruebas de video.

En el cuadro II se incluyen los resultados obtenidos de la aplicación de las red neuronales entrenadas con cada conjunto de datos sobre DS5, resaltando algunos de los mejores resultados en cada métrica seleccionada. Para estas pruebas, se tomaron los modelos entrenados con mejor AP_{50} de la evaluación anterior (sobre su propio conjunto de *test*).

Los resultados más destacables son los siguientes:

- El mejor AP_{50} se da en el DS2 con YOLOv5, mientras

Red neuronal	Dataset	AP	AP ₅₀	Tiempo (ms)
YOLOv5	DS1	0.164	0.599	19.3
	DS2	0.443	0.865	14.3
	DS3	0.320	0.707	19.7
	DS4	0.438	0.854	21.7
	DS7	0.437	0.836	15.9
YOLOv8	DS1	0.285	0.720	13.8
	DS2	0.503	0.853	4.9
	DS3	0.274	0.653	11.2
	DS4	0.474	0.850	10.9
	DS7	0.465	0.823	12.6
Faster-101	DS1	0.227	0.715	275.0
	DS2	0.352	0.783	282.0
	DS3	0.350	0.650	283.0
	DS4	0.279	0.622	295.0
	DS7	0.255	0.558	278.0
Faster-50	DS1	0.192	0.699	135.0
	DS2	0.356	0.773	139.0
	DS3	0.265	0.544	137.0
	DS4	0.323	0.708	145.0
	DS7	0.248	0.524	139.0

Cuadro II: Resultados exportados por cada modelo, evaluando sobre DS5. Se destacan los resultados del DS2 en las mejores redes neuronales y el AP₅₀ de DS4.

que el más bajo se da en MinneApple con Faster-50.

- El mejor AP se da con el DS2 en YOLOv8, mientras que el peor se da con DS1 y YOLOv5 (por el tamaño de las *bounding boxes* y que muchas manzanas no tienen etiquetas).
- El DS4 da mejores resultados sobre el DS5 que sobre su propio conjunto de *test* en todos los casos. Esto se debe a que el DS4 y DS5 son ambos tomados en el mismo lugar, pero el DS4 tiene imágenes con redes sobre los árboles, mientras que el DS5 no.
- Sorprendentemente, MinneApple se desempeña muy por debajo del DS2 a pesar de ser un subconjunto de este.
- Una de las razones por las que el AP no es muy bueno, es porque se detectan manzanas en árboles de la fila siguiente, que no están etiquetadas, aun con el DS4, que está entrenado con manzanas sin etiquetar en segunda fila. Esto baja la recuperación (*recall*), ya que agrega falsos positivos, disminuyendo el AP.

Respecto a tiempos de inferencia, YOLOv8 es más veloz que los demás modelos, seguido por YOLOv5, al que supera por menos de 10 ms en promedio. En el caso de Faster R-CNN, es sustancialmente más lento que ambas versiones de YOLO, dado que Faster-50 demora al menos 10 veces más y Faster-101 20 veces más.

A modo de resumen, los mejores resultados fueron obtenidos con YOLOv5 y YOLOv8, ambos entrenados con el DS2, por lo que son los modelos que se utilizan para la experimentación con los algoritmos de seguimiento.

IV-B. Conteo de manzanas

Para la experimentación de conteo se realizaron tres tipos de pruebas. La primera fue sobre videos etiquetados, evaluando distintas métricas de seguimiento. En el segundo tipo de prueba, se utilizaron videos completos sin etiquetar, conociendo únicamente el total de manzanas recolectadas de ese conjunto

de árboles. Finalmente, se realizan pruebas de regresión lineal con videos generados por el simulador, con el fin de evaluar la posibilidad de utilizar un coeficiente de ajuste para mejorar los resultados del seguimiento.

IV-B0a. Resultados sobre datos etiquetados: En esta etapa se evaluaron los algoritmos de seguimiento sobre el conjunto de datos DS-SIM, generado con el simulador, con 61 manzanas, y los conjuntos de datos DS-INIA-1 y DS-INIA-2, filmados en INIA Las Brujas, y con 139 y 200 manzanas respectivamente.

En lo que respecta a métricas de seguimiento, se tomaron un subconjunto de las definidas en el MOT Challenge [14]. Las métricas seleccionadas fueron *MOTA* [17], que toma en cuenta los errores en la predicción de la posición de los objetos y es una de las más utilizadas en el seguimiento de múltiples objetos, e *IDF1* que permite evaluar la conservación de los identificadores a lo largo del video, siendo una métrica global del conjunto de datos. También se seleccionó *HOTA* [37], por presentar mejor balance entre seguimiento y asignación de identificadores. Por último, se definió una métrica más simple, *IDs*, que consiste en la cantidad de identificadores asignados.

Como se puede notar en el cuadro III, los mejores resultados en *HOTA* y *MOTA* se dan con OCSort en todos los casos, mientras que los mejores resultados de *EAIDs* (error absoluto en la cantidad de *IDs*) son siempre menores con StrongSort, y los de *IDF1* varían entre StrongSort y OCSort. Esto induce a pensar que, si bien el seguimiento es mejor con OCSort, la asociación de los identificadores con los objetos rastreados es más exacta con StrongSort. En cuanto a tiempos de ejecución, OCSort y ByteTrack tienen tiempos similares, en el orden de los 30ms, mientras que StrongSort demora entre 3 y 4 veces más.

IV-B0b. Resultados sobre el recorrido completo: En segundo lugar, se buscó validar la cantidad de manzanas contadas en los videos filmados en INIA Las Brujas. Para esto, se analizó la relación entre las frutas visibles en los videos y las recolectadas.

Los resultados están disponibles en el cuadro IV, donde para cada modelo de detección se compara el rendimiento de los tres algoritmos de seguimiento seleccionados sobre los videos DS-INIA-IDA, DS-INIA-VUELTA y DS-INIA-COMPLETO. Los resultados de cada video están en las columnas “Ida”, “Vuelta” y “Recorrido completo”, y la columna “Suma” es la suma de las columnas “Ida” y “Vuelta”. Cabe mencionar que en el recorrido completo se incluye el giro del robot para cambiar de sentido, por lo que se detectan algunas manzanas adicionales.

En general, los resultados de cantidad total de manzanas son considerablemente mayores a la cantidad real de manzanas. Estos errores pueden darse por errores en el seguimiento de las manzanas, sin embargo, visualmente no parece que haya una gran variación en los identificadores asignados a las frutas. Lo que es más claro es que se están detectando una gran cantidad de manzanas de filas posteriores, lo que altera el conteo de la cantidad total de frutas.

Red neuronal	Dataset	Seguimiento	HOTA	MOTA	IDF1	EA IDs	Tiempo (ms)
YOLOv5 (DS2)	DS-SIM	OCSort	50.406	58.372	76.864	11	47.9
		Bytetrack	43.729	43.627	65.062	56	34.1
		Strongsort	47.471	55.052	72.471	1	177.8
	DS-INIA-1	OCSort	55.310	43.424	71.141	205	40.9
		Bytetrack	45.986	30.007	61.713	211	39.2
		Strongsort	54.39	28.676	69.612	151	170.9
	DS-INIA-2	OCSort	52.981	51.114	70.341	172	39.0
		Bytetrack	46.451	39.353	61.836	183	29.1
		Strongsort	50.762	37.248	68.020	96	154.6
YOLOv8 (DS2)	DS-SIM	OCSort	55.418	64.441	79.196	26	43.3
		Bytetrack	45.923	57.194	68.166	67	30.0
		Strongsort	52.069	61.728	74.927	1	175.0
	DS-INIA-1	OCSort	60.240	54.424	73.473	184	36.2
		Bytetrack	50.081	49.257	67.453	95	26.7
		Strongsort	59.301	53.316	73.832	69	134.2
	DS-INIA-2	OCSort	57.109	58.684	70.326	164	32.0
		Bytetrack	47.072	47.771	63.797	83	29.7
		Strongsort	56.086	55.377	71.031	26	118.9

Cuadro III: Resultados de algoritmos de seguimiento con métricas, donde *EA IDs* corresponde al error absoluto de cantidad de identificadores asignados. Se resaltan los mejores resultados de cada métrica.

Estas detecciones en el fondo son problemáticas dado su alto índice de confianza. Por esto, aplicar un filtro sobre esta métrica no solo no filtraría alguna de estas manzanas de otras filas, sino que descartaría algunas detecciones de la fila que se está evaluando. Algo similar ocurre con el filtrado por el tamaño de la *bounding box*, dado que algunas detecciones en el fondo tienen *bounding boxes* de mayor tamaño que otras en la fila actual.

Para resolver este problema, se exploraron diferentes enfoques para eliminar el fondo. Se intentaron utilizar herramientas para eliminar el fondo en cada cuadro, herramientas para generar una vista panorámica a partir del video, y finalmente una grabación capturada con una cámara estéreo ZED 2. En ningún caso los resultados fueron satisfactorios, ya que no se contaba con los recursos necesarios.

En lo que respecta a tiempos de ejecución por cuadro, los tiempos obtenidos muestran que StrongSort es el método más lento, con un tiempo promedio de 153ms para YOLOv8 y 97ms para YOLOv5. Esto es en el orden de 5 veces más lento que los otros dos modelos, donde ByteTrack es el más veloz, con un promedio de 22ms y OCSort logra en promedio 28ms.

RN	Seguimiento	Ida	Vuelta	Suma	Recorrido completo
YOLOv5 (DS2)	OCSort	2932	2789	5721	5782
	Bytetrack	2587	2503	5090	5121
	StrongSort	2659	2385	5044	5092
YOLOv8 (DS2)	OCSort	2621	2569	5190	5259
	Bytetrack	1877	1818	3695	3730
	StrongSort	1959	1808	3767	3810

Cuadro IV: Resultados de algoritmos de seguimiento sin etiquetas, donde la cantidad esperada de manzanas en la fila es 2133, “Ida”, “Vuelta” y “Recorrido completo” son los resultados de cada video y “Suma” es la suma de “Ida” y “Vuelta”.

IV-C. Ajuste mediante regresión lineal

Al contar la cantidad de manzanas en un recorrido completo, se pueden clasificar las manzanas en tres categorías según su

visibilidad [38]: visibles desde un único lado, desde ambos lados y no visibles desde afuera del árbol.

Dado que las manzanas visibles desde ambos y de ningún lado generan errores en el conteo mediante mecanismos de seguimiento, se plantea el uso de un método simple de regresión lineal para minimizar el error y predecir el valor real a partir del valor estimado.

Para construir métodos de ajuste, se generaron 20 instancias del simulador con parámetros constantes. Estas instancias contienen filas de 5 árboles, manteniendo las mismas condiciones ambientales, con un recorrido fijo a 0.2 m/s y posición de la cámara constante.

Además, se generan cinco instancias de prueba, las primeras tres con exactamente los mismos parámetros de las instancias de entrenamiento, resultando en 375 manzanas. En la cuarta instancia, se cambia la cantidad de árboles a 10 (722 manzanas), mientras que en la última se utilizan 3 filas de 5 árboles cada una (362 manzanas en la fila a estudiar), de forma de que aparezcan manzanas de fondo. En adelante, las instancias con 1 fila y 5 árboles se referencian como V1_5, las de 1 fila y 10 árboles V1_10, y V3_5 la instancia de 3 filas y 5 árboles en cada una.

Sobre los videos generados, se ejecutaron los algoritmos de seguimiento y detección definidos previamente (YOLOv5 y YOLOv8 combinados con StrongSort, Bytetrack y OCSort). Algunas de las ejecuciones se encuentran disponibles en una lista de reproducción pública del proyecto [39]. A partir de estas ejecuciones, se obtuvo una predicción de la cantidad de manzanas por instancia.

Los resultados obtenidos, combinados con la cantidad real de manzanas de cada video, se utilizaron como entrada de un modelo de regresión [40]. La recta de regresión permite ajustar nuevos resultados de predicciones, minimizando la diferencia con el valor real de la instancia.

Como método alternativo, se registraron los coeficientes de ajuste por cada combinación de modelos. La idea detrás de este método es encontrar un factor que permita ajustar las

predicciones de manera más simple, aplicando únicamente una multiplicación.

El coeficiente de ajuste es definido como el promedio de los cocientes de cada resultado esperado sobre su predicción. Esto se representa en la ecuación 1, donde n es la cantidad total de entradas del modelo.

$$\text{coeficiente} = \frac{1}{n} \sum_{i=1}^n \frac{\text{cantidad_real}}{\text{cantidad_predicha}} \quad (1)$$

IV-C1. Evaluación de métodos de ajuste: Para evaluar los modelos de ajuste, se ejecutaron los algoritmos en las instancias de prueba antes definidas y en el video del recorrido completo filmado (en adelante DS-INIA-COMPLETO).

Instancia	GT	R*CA	ER_{RCA}	Reg.	ER_{Reg}
V1_5	375	373	$0,06 \pm 0,04$	366	$0,04 \pm 0,03$
V1_10	722	672	$0,07 \pm 0,03$	491	$0,32 \pm 0,15$
V3_5	362	348	$0,04 \pm 0,02$	354	$0,02 \pm 0,09$
DS-INIA-COMPLETO	2133	2533	$0,20 \pm 0,19$	1261	$0,45 \pm 0,28$

Cuadro V: Resultados promedio de aplicar los modelos entrenados de ajuste a los resultados obtenidos, donde “R*CA” corresponde a los resultados de multiplicar el resultado por el coeficiente de ajuste, “Reg.” el ajuste con regresión lineal y “ER” los errores relativos en cada caso.

Los resultados promediados se incluyen en el cuadro V. Como se puede observar, los resultados difieren según el tipo de instancia evaluada. En el caso de V1_5 y V3_5, los errores relativos de ambos modelos se mantienen por debajo del 10 %. En estos casos, si bien los errores relativos son muy similares, el error cometido por el modelo de regresión es menor que el modelo de coeficiente, indicando un mejor desempeño del primero.

Sin embargo, en los resultados para la instancia V1_10, si bien el modelo de ajuste por coeficiente mantiene un error promedio cercano al 10 %, el modelo de regresión comete un error relativo promedio del 32 %. Esto se repite en el caso de DS-INIA-COMPLETO, donde el error relativo del modelo de regresión asciende al 45 % y el del modelo de coeficiente asciende a 20 %.

Cabe observar que, si bien en el video V3_5 hay tres filas de árboles, las manzanas de las filas posteriores a la fila grabada se ven muy pequeñas en comparación con las manzanas de la fila principal. Por este motivo, prácticamente no se detectan manzanas de otras filas, a diferencia del ambiente real. La diferencia puede apreciarse en la figura 7, donde se incluyen dos recortes de detecciones de videos del simulador y de DS-INIA-COMPLETO.

V. CONCLUSIONES

Durante la etapa de entrenamiento, se constató que la similitud de los datos de entrenamiento con los de evaluación influyen positivamente en los resultados. Por este motivo, el DS4, con apenas 80 imágenes, pero tomadas en INIA Las Brujas, obtiene resultados mejores que los de MinneApple (con 1000 imágenes) al evaluarlo sobre el DS5.



(a) Video simulado (b) Video de entorno real

Figura 7: Capturas de manzanas en filas posteriores de videos.

Otros factores determinantes son el tamaño y la variedad de los datos, lo que se visualiza en los resultados superiores de DS2 en comparación con DS7, siendo que el primero incluye al segundo y, por tanto, tiene mayor cantidad y variedad de imágenes.

A su vez, se concluyó que los conjuntos de datos etiquetados para recolección de manzanas son inadecuados para entrenar algoritmos de detección y conteo. Esto es relevante dada la mayor disponibilidad y facilidad de construcción de *datasets* para recolección.

Respecto a las redes neuronales para detección de objetos evaluadas, se concluye que YOLOv5 y YOLOv8 son superiores a Faster-101 y Faster-50, tanto en los resultados obtenidos como en los tiempos de entrenamiento y de detección. Concretamente, los modelos de Faster R-CNN evaluados tardan al menos 10 veces más que los de YOLO, llevando a tiempos de ejecución que no son suficientemente buenos para la ejecución en tiempo real.

Al comparar ambas versiones de YOLO, se obtienen resultados bastante similares, aunque YOLOv5 tiende a obtener mejores resultados de AP_{50} , mientras que YOLOv8 los obtiene en AP y con menores tiempos de ejecución. Se observa que esto puede estar afectado por el modelo de YOLOv8 seleccionado, siendo la versión *nano*, mientras que la versión de YOLOv5 seleccionada es la *small*. Aun teniendo esto en cuenta, se considera que YOLOv8 es la red neuronal con mejor desempeño y que convendría seguir estudiando.

Los algoritmos de seguimiento evaluados fueron StrongSort, ByteTrack y OCSort. De estos, OCSort es el que obtiene mejores resultados en general (tanto en MOTA como HOTA, y destacando en IDF1), pero StrongSort obtiene la menor cantidad de errores absolutos en detección. En cuanto a tiempos, ByteTrack es el más veloz, seguido de cerca por OCSort, y superando ampliamente a StrongSort, que tarda al menos 3 veces más.

Por otro lado, a la hora de evaluar sobre el recorrido completo, el algoritmo que se acerca más al valor real de cantidad de manzanas es StrongSort. Si bien esto es consistente con

sus valores de error absoluto de identificadores, los resultados de todos los algoritmos sobre este video se ven afectados por las manzanas detectadas en segunda fila y las condiciones de iluminación.

Por los motivos anteriores, se concluye que el mejor algoritmo de seguimiento es OCSort, siendo el más equilibrado en resultados obtenidos y tiempo de ejecución.

En lo que respecta a los modelos de ajuste, se destacan los buenos resultados de los métodos elegidos, logrando ajustar la estimación para tomar en cuenta las frutas que son visibles de ambos lados de la fila y las que no son visibles.

Sin embargo, estos resultados no parecen ser consistentes cuando las características de los datos evaluados difieren de las de los datos de entrenamiento. Particularmente, el modelo de regresión lineal se destaca cuando la cantidad de árboles se mantiene, con un error relativo menor al 10 %, pero aumenta al 30 % al tener el doble de árboles en el video evaluado. Esto no aplica para el método del coeficiente, que logra mantener el error relativo en el orden del 10 % incluso cuando se duplica la cantidad de árboles.

Por lo tanto, para evaluar predicciones sobre videos con una cantidad de árboles similar a la de los datos de entrenamiento, se recomienda el uso del modelo de regresión. Por el contrario, si se quiere variar la cantidad de árboles, se recomienda el uso del modelo del coeficiente.

REFERENCIAS

- [1] A. Kamilaris and F. X. Prenafeta-Boldú, "A review of the use of convolutional neural networks in agriculture," *The Journal of Agricultural Science*, vol. 156, no. 3, pp. 312–322, 2018.
- [2] M. K. van Ittersum and R. Rabbinge, "Concepts in production ecology for analysis and quantification of agricultural input-output combinations," *Field crops research*, vol. 52, no. 3, pp. 197–208, 1997.
- [3] Q. Wang, S. Nuske, M. Bergerman, and S. Singh, "Automated crop yield estimation for apple orchards," in *Experimental Robotics: The 13th International Symposium on Experimental Robotics*, pp. 745–758, Springer, 2013.
- [4] Z. Ge, S. Liu, F. Wang, Z. Li, and J. Sun, "Yolox: Exceeding yolo series in 2021," *arXiv preprint arXiv:2107.08430*, 2021.
- [5] G. Jocher, A. Chaurasia, and J. Qiu, "Yolo by ultralytics," 2023. Accedido: 2023-06-18.
- [6] J. Cao, X. Weng, R. Khirdkar, J. Pang, and K. Kitani, "Observation-centric sort: Rethinking sort for robust multi-object tracking," *arXiv preprint arXiv:2203.14360*, 2022.
- [7] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, "Bytetrack: Multi-object tracking by associating every detection box," in *Computer Vision—ECCV 2022: 17th European Conference, Tel Aviv, Israel, October 23–27, 2022, Proceedings, Part XXII*, pp. 1–21, Springer, 2022.
- [8] Y. Du, Z. Zhao, Y. Song, Y. Zhao, F. Su, T. Gong, and H. Meng, "Strong-sort: Make deepsort great again," *IEEE Transactions on Multimedia*, 2023.
- [9] S. Bargoti and J. Underwood, "Deep fruit detection in orchards," in *2017 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 3626–3633, IEEE, 2017.
- [10] B. Yan, P. Fan, X. Lei, Z. Liu, and F. Yang, "A real-time apple targets detection method for picking robot based on improved yolov5," *Remote Sensing*, vol. 13, no. 9, p. 1619, 2021.
- [11] W. Zhang, J. Wang, Y. Liu, K. Chen, H. Li, Y. Duan, W. Wu, Y. Shi, and W. Guo, "Deep-learning-based in-field citrus fruit detection and tracking," *Horticulture Research*, vol. 9, 2022.
- [12] P. Roy, W. Dong, and V. Isler, "Registering reconstructions of the two sides of fruit tree rows," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 1–9, IEEE, 2018.
- [13] M. Stein, S. Bargoti, and J. Underwood, "Image based mango fruit detection, localisation and yield estimation using multiple view geometry," *Sensors*, vol. 16, no. 11, p. 1915, 2016.
- [14] P. Dendorfer, A. Osep, A. Milan, K. Schindler, D. Cremers, I. Reid, S. Roth, and L. Leal-Taixé, "Motchallenge: A benchmark for single-camera multiple target tracking," *International Journal of Computer Vision*, vol. 129, pp. 845–881, 2021.
- [15] Y. Wu, A. Kirillov, F. Massa, W.-Y. Lo, and R. Girshick, "Detectron2," <https://github.com/facebookresearch/detectron2>, 2019.
- [16] Y. Wu, "Detectron2 model zoo and baselines," 2019.
- [17] M. Broström, "BoxMOT: A collection of SOTA real-time, multi-object trackers for object detectors," 2022.
- [18] F. C. Doerflinger, W. B. Miller, J. F. Nock, and C. B. Watkins, "Relationships between starch pattern indices and starch concentrations in four apple cultivars," *Postharvest Biology and Technology*, vol. 110, pp. 86–95, 2015.
- [19] Stanford Artificial Intelligence Laboratory, "Robotic operating system," 2007.
- [20] D. Cabrera, P. Rodríguez, and B. Carra, "Performance of different training systems in a pedestrian orchard for an efficient apple production," in *XII International Symposium on Integrating Canopy, Rootstock and Environmental Physiology in Orchard Systems 1346*, pp. 319–326, 2021.
- [21] A. Polgár, "GitHub - azazdeaz/fields-ignition: Generate random crop fields for Ignition Gazebo — github.com," 2020.
- [22] N. Koenig and A. Howard, "Design and use paradigms for gazebo, an open-source multi-robot simulator," in *2004 IEEE/RSJ international conference on intelligent robots and systems (IROS)*(IEEE Cat. No. 04CH37566), vol. 3, pp. 2149–2154, IEEE, 2004.
- [23] Blender Foundation, "Api overview 2014; blender python api." https://docs.blender.org/api/current/info_overview.html, 2014. Accedido: 2023-06-11.
- [24] OpenRobotics, "Costar husky sensor config 1," October 2021.
- [25] StereoLabs, "Zed2 stereo camera." Product information, 2020. URL: <https://www.stereolabs.com/zed2/>.
- [26] Roboflow, "Roboflow universe: Open source computer vision community." Accedido: 2024-04-09.
- [27] wangwang, "Apple dataset." <https://universe.roboflow.com/wangwang/apple-shybf>, Mayo 2022. Accedido: 2023-05-04.
- [28] N. Häni, P. Roy, and V. Isler, "Minneapolis: a benchmark dataset for apple detection and segmentation," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 852–858, 2020.
- [29] O. Girling, "Apple recognition dataset." <https://universe.roboflow.com/oliver-girling0-gmail-com/apple-recognition>, Enero 2023. Accedido: 2023-05-03.
- [30] Roboflow, "Generate augmented images to improve model performance," 2023. Accedido: 2023-06-07.
- [31] R. Automation. Accedido: 2024-04-10.
- [32] Supervisely, "Supervisely, unified os for computer vision," 2017. Accedido: 2023-06-07.
- [33] "Google colab," 2017. Accedido: 2023-05-31.
- [34] J. Terven and D. Cordova-Esparza, "A comprehensive review of yolo: From yolov1 to yolov8 and beyond," *arXiv preprint arXiv:2304.00501*, 2023.
- [35] SabrePC, "Epochs, batch size, iterations - how are they important to training ai and deep learning models." Accedido: 2024-04-10.
- [36] S. Lyu, R. Li, Y. Zhao, Z. Li, R. Fan, and S. Liu, "Green citrus detection and counting in orchards based on yolov5-cs and ai edge system," *Sensors*, vol. 22, no. 2, 2022.
- [37] J. Luiten, A. Osep, P. Dendorfer, P. Torr, A. Geiger, L. Leal-Taixé, and B. Leibe, "Hota: A higher order metric for evaluating multi-object tracking," *International journal of computer vision*, vol. 129, pp. 548–578, 2021.
- [38] O. Apolo-Apolo, J. Martínez-Guanter, G. Egea, P. Raja, and M. Pérez-Ruiz, "Deep learning techniques for estimation of the yield and size of citrus fruits using a uav," *European Journal of Agronomy*, vol. 115, p. 126030, 2020.
- [39] R. Garderes and F. Gutiérrez, "Videos: Construcción y evaluación de métodos de ajuste." https://www.youtube.com/watch?v=DpCvVA3c_qU&list=PLUL58q01TzjuGC-vsQkdnunfs36MG9vCt, 2023.
- [40] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.